

Realisierungsbericht

Auftraggeber Thomas Fehr
Projektleiter Mario Aeberhard
Autor Nicolas Ammeter, Mario Aeberhard, Loic Tobler
Klassifizierung *Nicht klassifiziert, Intern, Vertraulich, GEHEIM*
Status *In Arbeit, Genehmigt*

Änderungsverzeichnis

Datum	Version	Änderung	Autor
30.10.2025	V1.0	Initiale Erstellung	Nicolas Ammeter, Mario Aeberhard, Loic Tobler

Inhaltsverzeichnis

1	Zusammenfassung	3
2	Technische Detailspezifikation	3
2.1	Systemdesign.....	3
2.1.1	Struktur	3
2.1.2	Dynamik	5
2.2	Schnittstellendefinitionen.....	6
2.2.1	Die wichtigsten Events:	6
2.3	Datenmodell.....	8
2.4	Sicherheit (ISDS).....	9
3	Systemdokumentation.....	10
3.1	Anwendungshandbuch	10
3.1.1	Übersicht	10
3.1.2	Funktionen und Detailbeschreibung	10
3.1.3	Fehlerbehandlung	11
3.2	Integrations- und Installationshandbuch	11
3.3	Betriebshandbuch	12
4	Testspezifikation	13
4.1	Unit-Test	13
4.2	Systemtest	13

4.2.1	Testfälle	14
4.3	Testdurchführungen	18
4.3.1	Testfall 1 „Lobby erstellen“	18
4.3.2	Testfall 2 „Lobby beitreten – nicht existierende Lobby“	18
4.3.3	Testfall 3 „Lobby beitreten – Lobby voll“	18
4.3.4	Testfall 4 „Spiel starten“	18
4.3.5	Testfall 5 „Spielerbewegung“	18
4.3.6	Testfall 6 „Bombe setzen und Explosion“	19
5	Projektplanung	19
Anhang A: Quellcode		20
5.1.1	Index.ts	20
5.1.2	Main.tsx	35
5.1.3	App.tsx	36
5.1.4	Socket.ts	48
5.1.5	App.css	49
5.1.6	Index.css	57
5.1.7	Index.html	60
Anhang B: Testcode Unit-Tests		61

1 Zusammenfassung

Dieser Bericht dokumentiert die Realisierung von bomb.io, einem webbasierten Multiplayer-Spiel, das über socket.io in Echtzeit Spieleraktionen synchronisiert und eine dynamische Lobby-Struktur verwaltet. Im technischen Teil werden die wesentlichen Komponenten erläutert: das Client-Frontend in React, der Node.js-basierte Game-Server, die Echtzeitkommunikation über socket.io sowie die interne Modellierung von Lobbys, Spielern und Spielzuständen.

Das Systemdesign beschreibt die endgültige Architektur, bestehend aus klar getrennten Modulen für Lobby-Management, Spieler-Synchronisation und Spielregeln. Die Schnittstellenkapitel spezifizieren die eingesetzten socket.io-Events wie *lobby:create*, *lobby:update*, *match:start* und *player:move*.

Das Datenmodell umfasst die serverseitigen Objekte zur Verwaltung von Spielern, Lobbys und aktiven Matches, die während des Spiels im Arbeitsspeicher gehalten werden. Das Sicherheitskapitel erläutert die Umsetzung einer einfachen Zugriffskontrolle über Event-Validierung und serverseitige Zustandsprüfung.

Die Systemdokumentation beschreibt die Bedienung des Spiels, insbesondere das Erstellen und Beitreten von Lobbys sowie den Ablauf eines Matches. Die Testspezifikation definiert die wichtigsten Systemtests, darunter das Erstellen einer Lobby, das Beitreten weiterer Spieler und die korrekte Synchronisation der Spielzustände. Abschliessend verweist die Projektplanung auf die aktualisierten Aufwände der Realisierungsphase.

2 Technische Detailspezifikation

2.1 Systemdesign

2.1.1 Struktur

Das System bomb.io besteht aus zwei klar getrennten Hauptkomponenten: einem React-Frontend und einem Node.js-basierten Game-Server, der über socket.io die gesamte Echtzeitkommunikation verarbeitet. Persistente Daten werden nicht verwendet; sämtliche Spielzustände werden zur Laufzeit im Arbeitsspeicher des Servers verwaltet.

Die Architektur ist modular aufgebaut:

2.1.1.1 Frontend (React)

- **LobbyModule**
Verantwortlich für das Erstellen, Anzeigen und Aktualisieren von Lobbys. Nutzt socket.io-Client zum Empfangen von Lobby-Updates.
- **GameModule**
Rendert das Spielfeld, zeigt die Spielerpositionen und Bomben an.
- **SocketClient**
Abstraktionsschicht für alle Event-Listener und Emit-Aufrufe im Client.

2.1.1.2 Backend (Node.js + socket.io)

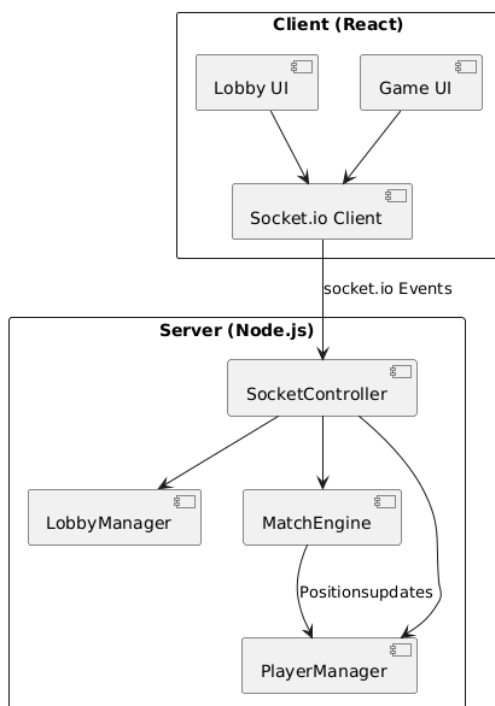
- **LobbyManager**
Verantwortlich für das Erstellen, Aktualisieren und Löschen von Lobbys. Hält interne Strukturen wie `lobbies: Map<lobbyId, LobbyObject>`.
- **PlayerManager**
Registriert Spieler, trackt Positionen und Status.

- **MatchEngine**
Verantwortlich für Spielregeln, Bombenlogik, Timer, Rundenauswertung.
- **SocketController**
Konkrete Event-Handler für:
 - lobby:create
 - lobby:join
 - lobby:update
 - match:start
 - player:move
 - bomb:place
- **StateStore (In-Memory)**
Temporäre Speicherung aller aktiven Spiele und Spielerzustände.

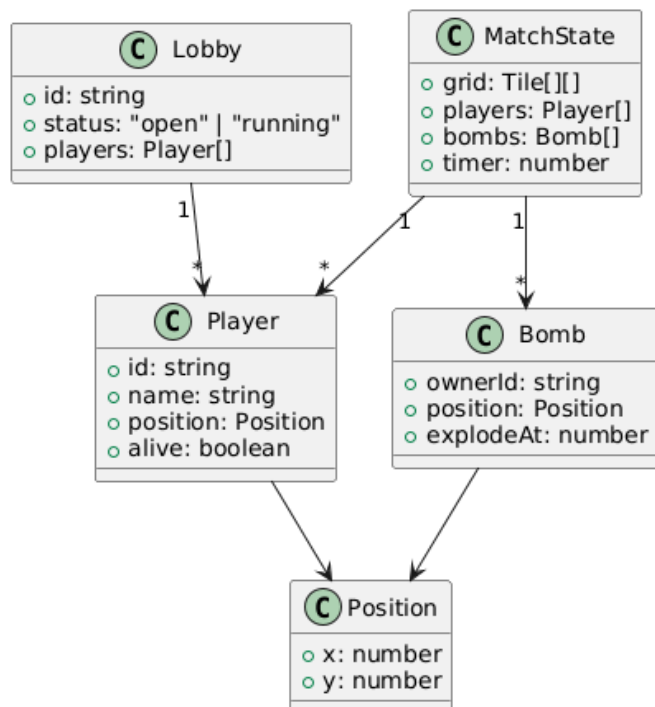
2.1.1.3 Designüberlegungen

- Es wurde bewusst auf eine Datenbank verzichtet, da bomb.io auf kurze, temporäre Echtzeitspiele ausgelegt ist.
- Alle Module sind streng getrennt, um Logik-Duplikate zu vermeiden (z. B. ist die Bombenlogik ausschliesslich in der MatchEngine implementiert).
- clientseitige Logik ist rein visuell; alle spielrelevanten Entscheidungen werden serverseitig validiert.

bomb.io - Systemarchitektur (Blockdiagramm)



bomb.io - Klassendiagramm



2.1.2 Dynamik

Die folgenden Abläufe gehören zu den zentralen Prozessen von bomb.io

Ablauf 1: Spieler erstellt eine Lobby

Beschreibung:

Der Client sendet lobby:create → Server erstellt Lobby → Server sendet lobby:update an alle Clients.

Wesentliche Schritte:

1. Spieler klickt „Lobby erstellen“.
2. React-Client sendet Event lobby:create.
3. LobbyManager.createLobby() erzeugt ein neues Lobby-Objekt.
4. SocketController sendet aktualisierte Liste an alle verbundenen Nutzer.
5. Client rendert neue Lobby.

Ablauf 2: Spieler bewegt sich im Spiel

(zentralster Echtzeitprozess)

Beschreibung:

Bewegungen werden vom Client gesendet, serverseitig überprüft und an alle anderen Spieler übertragen.

Schritte:

1. Spieler drückt Pfeiltaste.
2. Client sendet player:move mit Richtung.
3. MatchEngine berechnet neue Position (prüft Kollisionen).
4. Server sendet match:state mit aktualisierter Spielwelt.
5. Alle Clients updaten ihre Anzeige

Ablauf 3: Bombe platzieren

(enthält einfache Spielmechanik)

Schritte:

1. Spieler sendet bomb:place.
2. MatchEngine legt Bombe ins State-Objekt.
3. Timer läuft serverseitig.
4. Explosion triggert Kollisionsberechnung und Updates.
5. Server sendet match:state an all

2.2 Schnittstellendefinitionen

bomb.io nutzt ausschliesslich **socket.io-Events** als Kommunikationsschnittstelle.

2.2.1 Die wichtigsten Events:

2.2.1.1 Event: lobby:create

Richtung: Client → Server

Payload:

```
{  
  "playerName": "string"  
}
```

Bedeutung: Erzeugt eine neue Lobby und setzt den Spieler als Host.

2.2.1.2 Event: lobby:update

Richtung: Server → Client

Payload:

```
{  
  "lobbies": [  
    {  
      "id": "string",  
      "players": ["string"],  
      "status": "open" | "running"  
    }  
  ]  
}
```

Bedeutung: Aktuelle Liste aller Lobbys.

2.2.1.3 Event: match:start

Richtung: Server → Client

Startsignal für ein neues Spiel; enthält Startpositionen aller Spieler.

2.2.1.4 Event: player:move

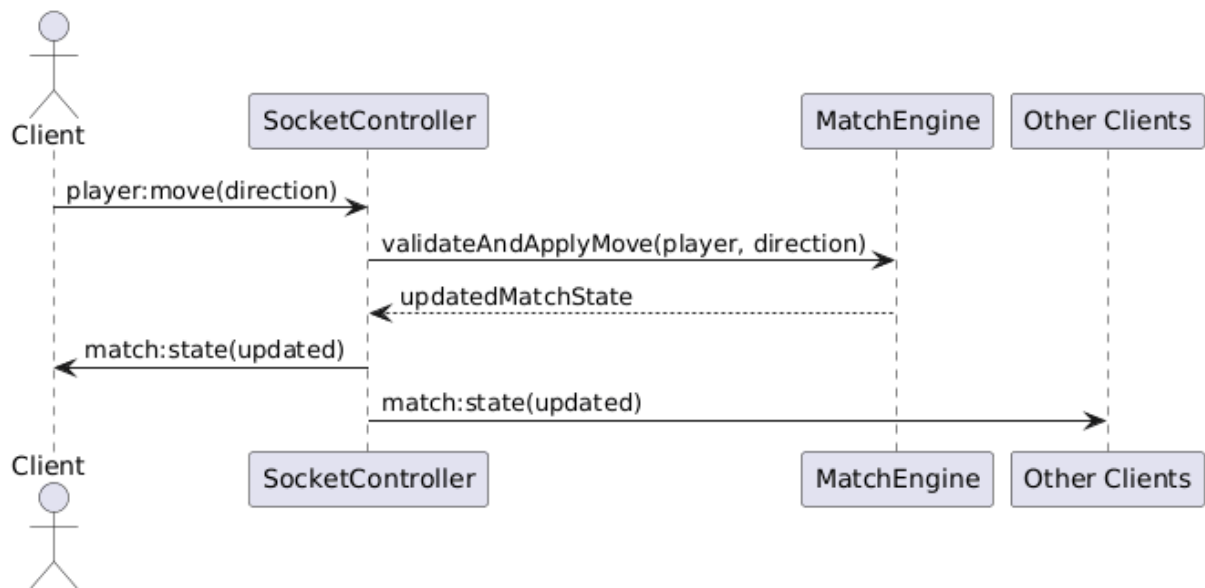
Richtung: Client → Server

Payload:

```
{ "direction": "up" | "down" | "left" | "right" }
```

Semantik: Der Server validiert die Bewegung und berechnet die neue Position.

bomb.io - Sequenzdiagramm: Spieler bewegt sich



2.2.1.5 Event: match:state

Richtung: Server → Client

Aktueller Zustand des Spielfelds, inklusive:

- Spielerpositionen
- Bomben
- Wände / Hindernisse
- Rundentimer

Zulässige Sequenzen

Ein Spieler darf:

- erst nach `lobby:join` → `match:start` Events senden
- während eines laufenden Matches keine Lobby-Events senden
- Bewegungen nur während `status = "running"` durchführen
- Der Server verhindert eventuelle Fehlsequenzen serverseitig vollständig.

2.3 Datenmodell

bomb.io verwendet keine Datenbank; alle Daten werden während eines Spiels im RAM gehalten. Das interne Modell besteht aus folgenden Kernobjekten:

Lobby

```
Lobby {  
  id: string  
  players: Player[]  
  status: "open" | "running"  
}
```

Player

```
Player {  
  id: string  
  name: string  
  position: { x: number, y: number }  
  alive: boolean  
}
```

MatchState

```
MatchState {  
  grid: Tile[][]  
  players: Player[]  
  bombs: Bomb[]  
  timer: number  
}
```

Bomb

```
Bomb {  
  ownerId: string  
  position: { x: number, y: number }  
  explodeAt: number  
}
```

Die Struktur genügt, da Daten persistent gespeichert werden müssen.

2.4 Sicherheit (ISDS)

Da bomb.io ein lokal gehostetes Spiel ohne personenbezogene Daten ist, sind die Anforderungen gering. Relevante Massnahmen:

Technisch:

- serverseitige Validierung jeder Aktion (Bewegungen, Bomben setzen, Lobbyaktionen)
- keine clientseitigen Entscheidungen über Spielregeln
- socket.io-Namespaces verhindern unautorisierte Events
- Schutz vor ungültigen Payloads (Typprüfung)

Organisatorisch:

- keine Speicherung von Nutzerdaten → keine Datenschutzrisiken
- Logdaten enthalten keine personenbezogenen Informationen

Für ein Schülerprojekt ist der Sicherheitsbedarf klein; kritische Risiken bestehen nicht.

3 Systemdokumentation

3.1 Anwendungshandbuch

3.1.1 Übersicht

bomb.io ist ein einfaches webbasiertes Mehrspieler-Spiel, das direkt im Browser ohne Installation gespielt werden kann. Die Spieler treten in einer kleinen Arena gegeneinander an, bewegen ihre Spielfigur und können Bomben setzen. Ziel ist es, die anderen Spieler auszuschalten und als Letzter übrig zu bleiben.

Bevor ein Spiel beginnt, erstellt ein Spieler eine Lobby oder tritt einer bestehenden Lobby bei. Sobald genügend Spieler anwesend sind, kann der Host das Spiel starten. Während des Spiels aktualisiert sich die Ansicht automatisch, da alle Bewegungen und Aktionen in Echtzeit übertragen werden.

3.1.2 Funktionen und Detailbeschreibung

Dieses Kapitel beschreibt alle Funktionen von bomb.io aus Sicht der Anwender. Die Erklärungen orientieren sich am typischen Ablauf vom Betreten der Webseite bis zum Ende eines Spiels. Alle Schritte sind bewusst einfach gehalten, damit auch nicht-technische Benutzer das System problemlos bedienen können.

Startseite

Beim Öffnen der Webseite erscheint die Startseite.

Der Benutzer hat zwei Hauptmöglichkeiten:

a) Lobby erstellen

Mit der Option „**Lobby erstellen**“ wird eine neue private Spielrunde eröffnet.

Nach der Erstellung erhält der Benutzer:

- eine **eindeutige Lobby-ID**
- die Möglichkeit, andere Spieler einzuladen
- die Übersicht aller bereits beigetretenen Spieler

Diese Lobby-ID wird benötigt, damit Freunde der Runde beitreten können.

b) Lobby beitreten

Wer bereits eine Lobby-ID von einem Freund erhalten hat, kann diese hier eingeben.

Nach dem Klick auf „**Beitreten**“ gelangt man direkt in die entsprechende Lobby.

Lobbyansicht

Die Lobby zeigt:

- die aktuelle **Spielerliste**
- den Status jedes Spielers (bereit/unbereit)
- den Host (Ersteller der Lobby)
- den Button „**Spiel starten**“ (nur für den Host sichtbar)
- einen Hinweis, wie weitere Spieler beitreten können

Die Lobby aktualisiert sich automatisch, sobald neue Spieler beitreten oder der Host das Spiel startet.

Spielstart

Sobald der Host den Startknopf drückt, wechseln alle Teilnehmer automatisch in das Spielfeld.

Wichtig für Anwender:

- Es ist kein erneutes Laden der Webseite nötig
- Der Übergang passiert automatisch
- Die Spieler erscheinen an zufälligen Startpositionen

Spielerbewegung

Der Spieler kann sich über die Tastatur bewegen:

- Pfeiltasten
- Bewegung erfolgt sofort und ohne Verzögerung

Die Bewegungen aller Spieler sind synchronisiert und erscheinen bei jedem zeitgleich.

3.1.3 Fehlerbehandlung

„Lobby nicht gefunden“

Ursache: Die Lobby wurde gelöscht oder ist gerade gestartet worden.

Vorgehen: Zurück zur Lobby-Übersicht gehen und eine andere Lobby wählen.

„Verbindung unterbrochen“

Ursache: Internetverbindung instabil oder Browserfenster wurde zu lange inaktiv.

Vorgehen: Seite neu laden. Der Spieler kann anschliessend wieder einer Lobby beitreten.

Spiel reagiert nicht mehr

Ursache: Browser hat zu viele Tabs offen oder das Gerät ist überlastet.

Vorgehen: Seite neu laden und möglichst andere Programme schliessen.

Tasten funktionieren nicht

Ursache: Browser-Fokus verloren (z. B. vorher UI angeklickt).

Vorgehen: Einmal ins Spielfeld klicken, danach erneut bewegen.

3.2 Integrations- und Installationshandbuch

bomb.io wird direkt im Webbrowser ausgeführt und benötigt keine Installation auf dem Endgerät des Anwenders. Das Spiel wird vollständig clientseitig geladen und über eine bestehende Serverinstanz betrieben. Der Anwender benötigt lediglich einen modernen Browser wie Chrome, Edge oder Firefox.

Voraussetzungen

- Stabiler Internetzugang
- Moderner Webbrowser (Chrome, Edge, Firefox, Safari)

- JavaScript aktiviert

Start des Systems (für Anwender)

1. Webadresse des Spiels im Browser öffnen.
2. Die Benutzeroberfläche lädt automatisch.
3. Der Nutzer kann direkt eine Lobby erstellen oder einer bestehenden Lobby beitreten.

Es sind keine weiteren Schritte erforderlich.

Das System ist sofort spielbereit.

3.3 Betriebshandbuch

Installation / Integration (für Betreiber des Systems)

Falls das System lokal oder auf einem Server betrieben wird:

1. Servercode starten (Node.js):
2. npm install
3. npm start
4. Den statischen Client (React-Build) bereitstellen, z. B. über:
 - Entwicklungsmodus: npm run dev
 - Produktionsmodus: npm run build und anschliessend deployen
5. Die URL der Client-Anwendung stellt das Spiel bereit.
6. Anwender verbinden sich ausschliesslich über den Browser.

4 Testspezifikation

4.1 Unit-Test

Für bomb.io wurden keine klassischen Unit-Tests umgesetzt. Der Grund dafür liegt in der Art des Systems: Ein grosser Teil der Spielmechanik basiert auf Echtzeitkommunikation über socket.io, bei der mehrere Clients gleichzeitig interagieren. Viele Funktionen sind direkt an den laufenden Spielzustand und an eingehende Events gebunden, wodurch isoliertes Testen einzelner Funktionen nur begrenzt aussagekräftig wäre.

Statt formalen Unit-Tests wurden folgende Massnahmen umgesetzt:

- Manuelle Modultests während der Entwicklung (z. B. direkte Überprüfung der Positionsberechnung oder Bombenlogik)
- Laufende Tests in der Entwicklungsumgebung mit zwei parallelen Browser-Clients
- Gezielte Test-Sessions nach funktionalen Änderungen (Smoke-Tests)

Die wichtigsten logischen Komponenten (Bewegungsvalidierung, Bombentimer, Kollisionslogik) wurden also nicht über ein Testframework, sondern über kontrollierte Spielsituationen und manuelles Debugging geprüft.

Da für bomb.io keine persistente Datenbank und keine statischen Businessregeln existieren, war dieser Ansatz für das Funktionsspektrum ausreichend.

4.2 Systemtest

Für bomb.io wurden die Systemtests vollständig manuell im Browser durchgeführt. Der Fokus lag darauf, die zentralen Anwendungsfälle des Spiels durchgehend zu prüfen und sicherzustellen, dass mehrere Clients korrekt in Echtzeit miteinander interagieren.

Da das System ohne Installation direkt im Browser läuft, wurden die Tests mit mindestens zwei parallelen Browserinstanzen (z. B. zwei Chrome-Fenster oder Chrome + Firefox) durchgeführt. Die Kommunikation erfolgte über eine laufende lokale Serverinstanz.

Ziele des Systemtests

- Sicherstellen, dass alle Spielabläufe von Lobby bis Spielende funktionieren
- Überprüfen der Echtzeitkommunikation zwischen mehreren Clients
- Validieren der Spielregeln (Bewegung, Bomben, Trefferlogik)
- Prüfen der Benutzeroberfläche (Darstellung, Aktualisierung, Übergänge)

Testumgebung

- Backend-Server lokal gestartet
- Zwei oder mehr Browserfenster auf dem gleichen Gerät
- Lokale Webadresse (z. B. <http://localhost>) geöffnet
- Standardtastatursteuerung für Bewegungen und Aktionen

Testmethode

Die Tests wurden von Hand durchgeführt.

Für jeden Anwendungsfall wurde die Ausgangslage hergestellt und anschliessend der komplette Ablauf beobachtet.

Die erwarteten Resultate wurden mit dem tatsächlichen Verhalten verglichen.

Sowohl grafische Anzeige als auch serverseitige Konsolenausgaben wurden berücksichtigt.

Getestete Bereiche

- Lobby-Management (Erstellen, Beitreten, Statuswechsel)
- Spielstart mit mehreren Teilnehmern
- Bewegungslogik inkl. Kollisionsbehandlung

- Bombenplatzierung, Countdown, Explosion
- Synchronisation der Spielwelt zwischen allen Clients
- Erkennung des Spielendes und Rückkehr zur Lobby

Die detaillierten Testfälle sind im nächsten Abschnitt (4.2.1) beschrieben.

4.2.1 Testfälle

Testfall 1 „Lobby erstellen“

Beschreibung	Ein Spieler erstellt erfolgreich eine neue Lobby.
Abgedeckte Anwendungsfälle	Lobby erstellen
Ausgangssituation	Es existiert noch keine Lobby und der Spieler befindet sich auf der Startseite.
Vorbereitungsschritte	1. Testdaten zurücksetzen (leere Lobby-Liste). 2. Browser (Firefox) auf dem Testgerät starten.
Testschritte	Erwartetes Resultat
1. Im Browser die URL bombio.notascam.ch öffnen.	Die Startseite von bomb.io wird angezeigt.
2. Den Button „Lobby erstellen“ wählen.	Eine neue Lobby wird erstellt und sichtbar.
3. Die angezeigte Lobby-ID prüfen.	Eine eindeutige Lobby-ID wird angezeigt.
4. Prüfen, ob der Spieler in der Spielerliste steht.	Der Spieler erscheint als Host in der Liste.

Testfall 2 „Lobby beitreten – nicht existierende Lobby“

Beschreibung	Ein Spieler versucht, einer nicht existierenden Lobby beizutreten.
Abgedeckte Anwendungsfälle	Lobby beitreten
Ausgangssituation	Der Spieler befindet sich auf der Startseite und es existiert keine Lobby mit der eingegebenen ID.
Vorbereitungsschritte	1. Leere Testdaten laden (keine Lobbys vorhanden). 2. Browser (Firefox) starten.
Testschritte	Erwartetes Resultat
1. bomb.io Webseite öffnen.	Die Startseite wird angezeigt.
2. Eine ungültige Lobby-ID eingeben, z. B. „XYZ123“.	Die ID wird übernommen.
3. Auf „Beitreten“ klicken.	Eine Fehlermeldung erscheint: „Lobby nicht gefunden“.
4. Bestätigen, dass keine Lobby geöffnet wird.	Der Spieler bleibt auf der Startseite.

Testfall 3 „Lobby beitreten – Lobby voll“

Beschreibung	Ein Spieler versucht, einer vollen Lobby beizutreten.
Abgedeckte Anwendungsfälle	Lobby beitreten, Spielerverwaltung
Ausgangssituation	Eine Lobby existiert bereits mit maximaler Spieleranzahl, z. B. 8 Spielern.
Vorbereitungsschritte	1. Testdaten laden: Lobby mit 8 Spielern erstellen. 2. Browser (Firefox) starten.
Testschritte	Erwartetes Resultat
1. bomb.io Webseite öffnen.	Die Startseite wird angezeigt.
2. Lobby-ID der vollen Lobby eingeben.	Die ID wird übernommen.
3. Auf „Beitreten“ klicken.	Fehlermeldung erscheint: „Lobby ist voll“.
4. Prüfen, dass kein Lobbybeitritt erfolgt.	Der Spieler bleibt auf der Startseite.

Testfall 4 „Spiel starten“

Beschreibung	Der Host startet das Spiel erfolgreich, nachdem sich zwei Spieler in derselben Lobby befinden.
Abgedeckte Anwendungsfälle	Spiel starten
Ausgangssituation	Mindestens zwei Spieler befinden sich in derselben Lobby.
Vorbereitungsschritte	Testdaten laden: Lobby mit zwei Spielern vorbereiten. Zwei Browserfenster (z. B. Firefox und Chrome) öffnen.
Testschritte	Erwartetes Resultat
Die Lobby im ersten Browser öffnen	Die Lobby wird korrekt angezeigt.
Im zweiten Browser der Lobby beitreten. Der zweite Spieler erscheint in der Spielerliste.	Im zweiten Browser der Lobby beitreten. Der zweite Spieler erscheint in der Spielerliste.
Im ersten Browser den Button „Starten“ wählen	Beide Clients wechseln ins Spielfeld.

Prüfen, ob Startpositionen angezeigt werden	Beide Spieler erscheinen korrekt im Spielfeld.
---	--

Testfall 5 „Spielerbewegung“

Beschreibung	Ein Spieler bewegt seine Figur, und die Bewegung wird in Echtzeit synchronisiert.
Abgedeckte Anwendungsfälle	Bewegungslogik, Echtzeitkommunikation
Ausgangssituation	Das Spiel läuft und beide Spieler befinden sich im Spielfeld.
Vorbereitungsschritte	Spiel wie im vorherigen Testfall starten. Beide Browserfenster sichtbar nebeneinander anordnen.
Testschritte	Erwartetes Resultat
Im ersten Browser eine Pfeiltaste drücken	Die Figur bewegt sich sichtbar.
Im zweiten Browser die Bewegung beobachten	Bewegung erscheint gleichzeitig im zweiten Fenster.
Gegen eine Wand laufen	Figur bleibt stehen; keine Positionsänderung.
Mehrere schnelle Bewegungen durchführen	Alle Bewegungen werden ohne Verzögerung synchronisiert.

Testfall 6 „Bombe setzen und Explosion“

Beschreibung	Ein Spieler setzt eine Bombe, die nach Ablauf des Timers explodiert.
Abgedeckte Anwendungsfälle	Bomben setzen, Explosionslogik
Ausgangssituation	Das Spiel läuft und der Spieler befindet sich auf freier Fläche.
Vorbereitungsschritte	Spiel starten und in geeigneter Position warten. Keine anderen Explosionen abwarten.
Testschritte	Erwartetes Resultat
Leertaste drücken, um eine Bombe zu setzen	Die Bombe erscheint unter dem Spieler.
Explosion abwarten	Explosion erfolgt nach Ablauf des Timers.
Prüfen, ob zerstörbare Blöcke verschwinden	Betroffene Blöcke werden korrekt entfernt.
Prüfen, ob Spieler getroffen wird	Spieler wird eliminiert, falls in Explosionsradius.
Darstellung in beiden Clients prüfen	Explosion und Änderungen werden synchron angezeigt.

1.

4.3 Testdurchführungen

4.3.1 Testfall 1 „Lobby erstellen“

Tester: **Nicolas Ammeter**Datum: **13.11.2025**

Testschritt	Erfüllt	Bemerkung
1.	Ja	
2.	Ja	
3.	Ja	
4.	Ja	

4.3.2 Testfall 2 „Lobby beitreten – nicht existierende Lobby“

Tester: **Mario Aeberhard**Datum, Zeit: **18.11.2025**

Ganzer Testfall erfüllt. (Tabelle mit Testschritten erübrigt sich.)

4.3.3 Testfall 3 „Lobby beitreten – Lobby voll“

Tester: **Loic Tobler**Datum, Zeit: **20.11.2025**

Ganzer Testfall erfüllt. (Tabelle mit Testschritten erübrigt sich.)

4.3.4 Testfall 4 „Spiel starten“

Tester: **Nicolas Ammeter**Datum, Zeit: **25.11.2025**

Testschritt	Erfüllt	Bemerkung
1.	Ja	
2.	Ja	
3.	Ja	
4.	Ja	

4.3.5 Testfall 5 „Spielerbewegung“

Tester: **Mario Aeberhard**Datum, Zeit: **26.11.2025**

Testschritt	Erfüllt	Bemerkung
1.	Ja	
2.	Ja	
3.	Ja	
4.	Ja	

4.3.6 Testfall 6 „Bombe setzen und Explosion“Tester: **Nicolas Ammeter**Datum, Zeit: **27.11.2025**

Testschritt	Erfüllt	Bemerkung
1.	Ja	
2.	Ja	
3.	Ja	
4.	Ja	

5 Projektplanung

Siehe Projektplan, Version 3.0 vom 27.11.2025

Anhang A: Quellcode

Aufgrund der Länge, Formatierungsgrenzen von Word und der Lesbarkeit wird ausdrücklich empfohlen, den Code direkt über das GitHub-Repository einzusehen.

<https://github.com/DerHagelmacher/bomb.io>

Nr.	Pfad / Datei-name	Sprache / Typ	Kurzbeschreibung
A1	server/src/index.ts	TypeScript	Express- und Socket.io-Server, Spiel-Logik
A2	client/src/main.tsx	TypeScript	React-Einstieg, Root-Render
A3	client/src/App.tsx	TypeScript	Lobby, Match-Ansicht, Bomben-Logik
A4	client/src/socket.ts	TypeScript	Socket.io-Client, Verbindungsaufbau
A5	client/src/App.css	CSS	Styling für Lobby, Match, Neon-Layout
A6	client/src/index.css	CSS	Globales Styling (Body, Root, Variablen)
A7	client/index.html	HTML	Basis-HTML für Vite/React

5.1.1 Index.ts

```
import express from "express";
import http from "http";
import { Server } from "socket.io";
import cors from "cors";
```

```
type Player = {
  id: string;      // socket.id
  nickname: string;
  ready: boolean;
  x: number;
  y: number;
  alive: boolean;
  lastMoveAt: number;
  wins: number;
};
```

```
type Lobby = {
  id: string;
  ownerId: string;
  players: Record<string, Player>;
  maxPlayers: number;
```

```
createdAt: number;
started: boolean;
endAt?: number;

bombHolderId?: string | null;
bombEndAt?: number | null;

round: number;
eliminationOrder: string[];
};

const FIELD_WIDTH = 400;
const FIELD_HEIGHT = 300;

const MATCH_DURATION_MS = 2 * 60 * 1000; // Sicherheitsende nach 2 Minuten
const BOMB_DURATION_MS = 15_000;         // 15 Sekunden Fixdauer pro Bombe
const BOMB_TOUCH_RADIUS = 20;            // Hitbox-Radius in Pixel
const AFK_THRESHOLD_MS = 3_000;          // ab 3s Stillstand gilt AFK

const lobbies = new Map<string, Lobby>();
const lobbyIntervals = new Map<string, NodeJS.Timeout>();

function makeLobbyId(): string {
  return Math.random().toString(36).slice(2, 8).toUpperCase();
}

function alivePlayers(lobby: Lobby): Player[] {
  return Object.values(lobby.players).filter((p) => p.alive);
}

function lobbyToDTO(lobby: Lobby) {
  return {
    id: lobby.id,
    ownerId: lobby.ownerId,
    started: lobby.started,
    maxPlayers: lobby.maxPlayers,
```

```
round: lobby.round,  
players: Object.values(lobby.players).map((p) => ({  
  id: p.id,  
  nickname: p.nickname,  
  ready: p.ready,  
  alive: p.alive,  
  wins: p.wins,  
})),  
};  
}
```

```
function sendLobby(io: Server, lobbyId: string) {  
  const lobby = lobbies.get(lobbyId);  
  if (!lobby) return;  
  io.to(lobbyId).emit("lobby:update", lobbyToDTO(lobby));  
}
```

```
function sendMatchState(io: Server, lobby: Lobby) {  
  if (!lobby.started || !lobby.endAt) return;  
  const remainingMs = Math.max(0, lobby.endAt - Date.now());
```

```
  const bomb =  
    lobby.bombHolderId && lobby.bombEndAt  
    ? {  
      holderId: lobby.bombHolderId,  
      remainingMs: Math.max(0, lobby.bombEndAt - Date.now()),  
    }  
    : undefined;
```

```
  io.to(lobby.id).emit("match:state", {  
    lobbyId: lobby.id,  
    remainingMs,  
    players: alivePlayers(lobby).map((p) => ({  
      id: p.id,  
      nickname: p.nickname,  
      x: p.x,
```

```
y: p.y,  
))),  
bomb,  
});  
}
```

```
function cleanupLobbyIfEmpty(lobbyId: string) {  
  const lobby = lobbies.get(lobbyId);  
  if (!lobby) return;  
  if (Object.keys(lobby.players).length === 0) {  
    console.log("[cleanup] Lobby gelöscht:", lobbyId);  
    const interval = lobbyIntervals.get(lobbyId);  
    if (interval) {  
      clearInterval(interval);  
      lobbyIntervals.delete(lobbyId);  
    }  
    lobbies.delete(lobbyId);  
  }  
}
```

```
/**  
 * Bombe ggf. weitergeben, wenn Bombentraeger einen anderen Spieler beruehrt.  
 * WICHTIG: bombEndAt wird NICHT zurueckgesetzt – Timer laeuft weiter.  
 */
```

```
function tryTransferBomb(lobby: Lobby, moverId: string) {  
  if (!lobby.bombHolderId) return;  
  if (lobby.bombHolderId !== moverId) return;  
  
  const holder = lobby.players[moverId];  
  if (!holder || !holder.alive) return;  
  
  const others = alivePlayers(lobby).filter((p) => p.id !== moverId);  
  
  for (const other of others) {  
    const dx = other.x - holder.x;  
    const dy = other.y - holder.y;
```

```
const dist = Math.hypot(dx, dy);

if (dist <= BOMB_TOUCH_RADIUS) {
  lobby.bombHolderId = other.id;
  console.log("[bomb] transfer", holder.nickname, "→", other.nickname);
  return;
}
}
}

/**
 * Explosion der Bombe: Spieler fliegt raus, ggf. neuer Bombentraeger oder Winner.
 */
function handleBombExplosion(
  io: Server,
  lobby: Lobby,
  now: number,
  interval: NodeJS.Timeout
) {
  const holderId = lobby.bombHolderId;
  if (!holderId) return;

  const loser = lobby.players[holderId];
  if (loser && loser.alive) {
    loser.alive = false;
    loser.ready = false;
    lobby.eliminationOrder.push(loser.id);

    io.to(lobby.id).emit("bomb:exploded", {
      lobbyId: lobby.id,
      loserId: loser.id,
      loserName: loser.nickname,
    });

    console.log("[bomb] exploded on:", loser.nickname);
  }
}
```



```
const alive = alivePlayers(lobby);

// Winner?
if (alive.length <= 1) {
  lobby.started = false;

  const winner = alive[0] ?? null;
  if (winner) {
    winner.wins += 1;
  }

  const standingsIds: string[] = [];
  if (winner) standingsIds.push(winner.id);
  standingsIds.push(...lobby.eliminationOrder.slice().reverse());

  const standings = standingsIds.map((id, index) => {
    const p = lobby.players[id];
    return {
      id,
      nickname: p?.nickname ?? "???",
      place: index + 1,
      wins: p?.wins ?? 0,
    };
  });

  io.to(lobby.id).emit("match:ended", {
    lobbyId: lobby.id,
    reason: "winner",
    winnerId: winner?.id ?? null,
    winnerName: winner?.nickname ?? null,
    standings,
  });

  // ----- Auto-Reset fuer naechste Runde -----
  for (const p of Object.values(lobby.players)) {
```

```
p.alive = true;
p.ready = false;
p.x = FIELD_WIDTH / 2;
p.y = FIELD_HEIGHT / 2;
}
lobby.bombHolderId = null;
lobby.bombEndAt = null;
lobby.eliminationOrder = [];
lobby.started = false;
lobby.endAt = undefined;

clearInterval(interval);
lobbyIntervals.delete(lobby.id);
sendLobby(io, lobby.id);
return;
}

// Neue Bombe an zufaelligen lebenden Spieler, Timer neu 15s
const next = alive[Math.floor(Math.random() * alive.length)];
lobby.bombHolderId = next.id;
lobby.bombEndAt = now + BOMB_DURATION_MS;

console.log("[bomb] new holder after explosion:", next.nickname);

sendMatchState(io, lobby);
}

const app = express();
app.use(cors({ origin: "http://localhost:5173" }));
app.use(express.json());

app.get("/health", (_req, res) => res.json({ ok: true }));

const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: "http://localhost:5173" },
```

```
});
```

```
io.on("connection", (socket) => {  
  console.log("Neuer Client:", socket.id);
```

```
  socket.onAny((event, ...args) => {  
    console.log("[onAny]", socket.id, "event:", event, "data:", args);  
  });
```

```
// Lobby erstellen
```

```
socket.on(  
  "lobby:create",  
  ({ nickname, maxPlayers = 4 }: { nickname: string; maxPlayers?: number }) => {  
    if (!nickname || !nickname.trim()) {  
      socket.emit("lobby:error", "Nickname noetig");  
      return;  
    }  
  }
```

```
  const id = makeLobbyId();  
  const lobby: Lobby = {  
    id,  
    ownerId: socket.id,  
    maxPlayers: Math.max(2, Math.min(8, Number(maxPlayers) || 4)),  
    createdAt: Date.now(),  
    started: false,  
    players: {},  
    round: 0,  
    eliminationOrder: [],  
  };
```

```
  lobbies.set(id, lobby);  
  socket.join(id);
```

```
  lobby.players[socket.id] = {  
    id: socket.id,  
    nickname: nickname.trim(),
```

```
    ready: false,
    x: FIELD_WIDTH / 2,
    y: FIELD_HEIGHT / 2,
    alive: true,
    lastMoveAt: Date.now(),
    wins: 0,
  };

  socket.emit("lobby:created", { lobbyId: id });
  sendLobby(io, id);
}
);

// Lobby beitreten
socket.on(
  "lobby:join",
  ({ lobbyId, nickname }: { lobbyId: string; nickname: string }) => {
    const id = lobbyId?.toUpperCase();
    const lobby = lobbies.get(id);
    if (!lobby) return socket.emit("lobby:error", "Lobby nicht gefunden");
    if (lobby.started) return socket.emit("lobby:error", "Spiel bereits gestartet");
    if (Object.keys(lobby.players).length >= lobby.maxPlayers) {
      return socket.emit("lobby:error", "Lobby ist voll");
    }

    socket.join(lobby.id);
    const existing = lobby.players[socket.id];

    lobby.players[socket.id] = {
      id: socket.id,
      nickname: nickname?.trim() || existing?.nickname || "Player",
      ready: false,
      x: FIELD_WIDTH / 2,
      y: FIELD_HEIGHT / 2,
      alive: true,
      lastMoveAt: Date.now(),
```

```
wins: existing?.wins ?? 0,
};

sendLobby(io, lobby.id);
}
);

// Ready toggeln
socket.on(
  "lobby:ready",
  ({ lobbyId, ready }: { lobbyId: string; ready: boolean }) => {
    const lobby = lobbies.get(lobbyId);
    if (!lobby) return;
    const p = lobby.players[socket.id];
    if (!p) return;
    p.ready = !ready;
    sendLobby(io, lobby.id);
  }
);

// Lobby verlassen
socket.on("lobby:leave", ({ lobbyId }: { lobbyId: string }) => {
  const lobby = lobbies.get(lobbyId);
  if (!lobby) return;
  const p = lobby.players[socket.id];
  if (!p) return;

  if (lobby.started && p.alive) {
    p.alive = false;
    lobby.eliminationOrder.push(p.id);
  }

  delete lobby.players[socket.id];
  socket.leave(lobbyId);

  if (lobby.ownerId === socket.id) {
```

```
    const next = Object.keys(lobby.players)[0];
    if (next) lobby.ownerId = next;
  }

  sendLobby(io, lobby.id);
  cleanupLobbyIfEmpty(lobby.id);
});

// Match starten
socket.on("match:start", ({ lobbyId }: { lobbyId: string }) => {
  const lobby = lobbies.get(lobbyId);
  if (!lobby) return socket.emit("lobby:error", "Lobby nicht gefunden");
  if (lobby.ownerId !== socket.id) {
    return socket.emit("lobby:error", "Nur Host kann starten");
  }

  const playersArr = alivePlayers(lobby);
  const readyArr = playersArr.filter((p) => p.ready);

  if (readyArr.length < 2) {
    return socket.emit("lobby:error", "Mindestens 2 bereite Spieler noetig");
  }

  const centerX = FIELD_WIDTH / 2;
  const centerY = FIELD_HEIGHT / 2;
  const radius = 80;

  playersArr.forEach((p, index) => {
    const angle = (index / playersArr.length) * Math.PI * 2;
    p.x = centerX + Math.cos(angle) * radius;
    p.y = centerY + Math.sin(angle) * radius;
    p.alive = true;
    p.lastMoveAt = Date.now();
  });

  lobby.round += 1;
```

```
lobby.started = true;

lobby.endAt = Date.now() + MATCH_DURATION_MS;
lobby.eliminationOrder = [];

// Zufälliger erster Bombentraeger
const randomIndex = Math.floor(Math.random() * playersArr.length);
lobby.bombHolderId = playersArr[randomIndex].id;
lobby.bombEndAt = Date.now() + BOMB_DURATION_MS;

io.to(lobby.id).emit("match:started", {
  lobbyId: lobby.id,
  t: Date.now(),
  durationMs: MATCH_DURATION_MS,
  round: lobby.round,
});

const oldInterval = lobbyIntervals.get(lobby.id);
if (oldInterval) clearInterval(oldInterval);

const interval = setInterval(() => {
  const currentLobby = lobbies.get(lobby.id);
  if (!currentLobby || !currentLobby.started || !currentLobby.endAt) {
    clearInterval(interval);
    lobbyIntervals.delete(lobby.id);
    return;
  }
},

const now = Date.now();
const remaining = currentLobby.endAt - now;

if (remaining <= 0) {
  currentLobby.started = false;
  sendMatchState(io, currentLobby);
  io.to(currentLobby.id).emit("match:ended", {
    lobbyId: currentLobby.id,
    reason: "time",
```

```
});

// Auto-Reset auch bei Zeitablauf
for (const p of Object.values(currentLobby.players)) {
  p.alive = true;
  p.ready = false;
  p.x = FIELD_WIDTH / 2;
  p.y = FIELD_HEIGHT / 2;
}
currentLobby.bombHolderId = null;
currentLobby.bombEndAt = null;
currentLobby.eliminationOrder = [];
currentLobby.started = false;
currentLobby.endAt = undefined;

clearInterval(interval);
lobbyIntervals.delete(currentLobby.id);
sendLobby(io, currentLobby.id);
return;
}

if (currentLobby.bombHolderId && currentLobby.bombEndAt) {
  const holder = currentLobby.players[currentLobby.bombHolderId];
  let bombRemaining = currentLobby.bombEndAt - now;

  if (holder && holder.alive) {
    const idleMs = now - holder.lastMoveAt;
    if (idleMs > AFK_THRESHOLD_MS) {
      // AFK → Bombe tickt schneller (einfacher Speedup)
      bombRemaining -= 1000;
    }
  }
}

if (bombRemaining <= 0) {
  handleBombExplosion(io, currentLobby, now, interval);
  return;
}
```



```
    }  
  }  
  
  sendMatchState(io, currentLobby);  
}, 1000);  
  
lobbyIntervals.set(lobby.id, interval);  
sendMatchState(io, lobby);  
sendLobby(io, lobby.id);  
});  
  
// Spieler-Bewegung  
socket.on(  
  "player:move",  
  ({ lobbyId, dx, dy }: { lobbyId: string; dx: number; dy: number }) => {  
    const lobby = lobbies.get(lobbyId);  
    if (!lobby || !lobby.started) return;  
    const p = lobby.players[socket.id];  
    if (!p || !p.alive) return;  
  
    p.x += dx;  
    p.y += dy;  
    p.lastMoveAt = Date.now();  
  
    p.x = Math.max(0, Math.min(FIELD_WIDTH, p.x));  
    p.y = Math.max(0, Math.min(FIELD_HEIGHT, p.y));  
  
    tryTransferBomb(lobby, socket.id);  
    sendMatchState(io, lobby);  
  }  
);  
  
// Disconnect  
socket.on("disconnect", () => {  
  console.log("Client weg:", socket.id);
```

```
for (const lobby of lobbies.values()) {
  if (lobby.players[socket.id]) {
    const wasOwner = lobby.ownerId === socket.id;
    const p = lobby.players[socket.id];

    if (lobby.started && p.alive) {
      p.alive = false;
      lobby.eliminationOrder.push(p.id);
    }

    delete lobby.players[socket.id];

    if (wasOwner) {
      const next = Object.keys(lobby.players)[0];
      if (next) lobby.ownerId = next;
    }

    sendLobby(io, lobby.id);
    cleanupLobbyIfEmpty(lobby.id);
  }
}

});

});

const PORT = 3001;
server.listen(PORT, () => {
  console.log(`Server laeuft auf Port ${PORT}`);
});
```

5.1.2 Main.tsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.tsx'

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

5.1.3 App.tsx

```
import { useEffect, useMemo, useState, useRef } from "react";
import { socket } from "../socket";
import "../App.css";

type PlayerDTO = {
  id: string;
  nickname: string;
  ready: boolean;
  alive: boolean;
  wins: number;
};

type LobbyDTO = {
  id: string;
  ownerId: string;
  started: boolean;
  maxPlayers: number;
  round: number;
  players: PlayerDTO[];
};

type MatchPlayer = { id: string; nickname: string; x: number; y: number };
type MatchState = {
  lobbyId: string;
  remainingMs: number;
  players: MatchPlayer[];
  bomb?: {
    holderId: string;
    remainingMs: number;
  };
};

type Standing = {
  id: string;
  nickname: string;
  place: number;
  wins: number;
};

type WinnerScreenState = {
  lobbyId: string;
  reason: string;
  winnerId: string | null;
  winnerName: string | null;
  standings: Standing[];
};

type KillFeedItem = { id: number; text: string; ts: number };

const MOVE_SPEED = 0.25; // Pixel pro ms

export default function App() {
  const [nickname, setNickname] = useState("");
  const [lobbyId, setLobbyId] = useState("");
```

```
const [lobby, setLobby] = useState<LobbyDTO | null>(null);
const [error, setError] = useState<string | null>(null);
const [matchState, setMatchState] = useState<MatchState | null>(null);
const [winnerScreen, setWinnerScreen] = useState<WinnerScreenState | null>(
  null
);
const [killFeed, setKillFeed] = useState<KillFeedItem[]>([]);

// Socket-Events
useEffect(() => {
  const onConnect = () => console.log("Verbunden:", socket.id);

  const onLobbyCreated = ({ lobbyId }: { lobbyId: string }) =>
    setLobbyId(lobbyId);

  const onLobbyUpdate = (data: LobbyDTO) => {
    setLobby(data);
    setError(null);
  };

  const onLobbyError = (msg: string) => setError(msg);

  const onMatchStarted = (data: {
    lobbyId: string;
    t: number;
    durationMs: number;
    round: number;
  }) => {
    console.log("match:started", data);
    setWinnerScreen(null);
  };

  const onMatchState = (state: MatchState) => {
    setMatchState(state);
  };

  const onBombExploded = (data: {
    lobbyId: string;
    loserId: string;
    loserName: string;
  }) => {
    const now = Date.now();
    const text = `💣 ${data.loserName} wurde eliminiert`;
    setKillFeed((prev) =>
      [...prev, { id: now, text, ts: now }].slice(-5)
    );
  };

  const onMatchEnded = (data: {
    lobbyId: string;
    reason?: string;
    winnerId?: string | null;
    winnerName?: string | null;
    standings?: Standing[];
  }) => {
    console.log("match:ended", data);
```

```
setMatchState(null);

if (data.reason === "winner" && data standings) {
  setWinnerScreen({
    lobbyId: data.lobbyId,
    reason: data.reason,
    winnerId: data.winnerId ?? null,
    winnerName: data.winnerName ?? null,
    standings: data standings,
  });
} else if (data.reason === "time") {
  setError("Match zu Ende – Zeit abgelaufen");
} else {
  setError("Match beendet");
}
};

socket.on("connect", onConnect);
socket.on("lobby:created", onLobbyCreated);
socket.on("lobby:update", onLobbyUpdate);
socket.on("lobby:error", onLobbyError);
socket.on("match:started", onMatchStarted);
socket.on("match:state", onMatchState);
socket.on("bomb:exploded", onBombExploded);
socket.on("match:ended", onMatchEnded);

return () => {
  socket.off("connect", onConnect);
  socket.off("lobby:created", onLobbyCreated);
  socket.off("lobby:update", onLobbyUpdate);
  socket.off("lobby:error", onLobbyError);
  socket.off("match:started", onMatchStarted);
  socket.off("match:state", onMatchState);
  socket.off("bomb:exploded", onBombExploded);
  socket.off("match:ended", onMatchEnded);
};
}, []);

const isOwner = useMemo(() => lobby?.ownerId === socket.id, [lobby]);
const me = useMemo(
  () => lobby?.players.find((p) => p.id === socket.id) || null,
  [lobby]
);

const inMatch = !(lobby && lobby.started && matchState);
const myMatchPlayer =
  matchState?.players.find((p) => p.id === socket.id) || null;
const isSpectator = !(inMatch && matchState && !myMatchPlayer);

// Killfeed: nur Einträge der letzten 6 Sekunden
const visibleKillFeed = useMemo(() => {
  const now = Date.now();
  return killFeed.filter((item) => now - item.ts < 6000);
}, [killFeed, matchState]);

// Refs statt state-Abhängigkeit
```

```
const pressedKeysRef = useRef<Set<string>>(new Set());
const lobbyIdRef = useRef<string | null>(null);
```

```
// Lobby-ID immer aktuell halten
useEffect(() => {
  if (matchState?.lobbyId) {
    lobbyIdRef.current = matchState.lobbyId;
  }
}, [matchState?.lobbyId]);
```

```
// Smooth movement loop
useEffect(() => {
  if (!inMatch) {
    pressedKeysRef.current.clear();
    return;
  }
}
```

```
const keys = pressedKeysRef.current;
```

```
function onKeyDown(e: KeyboardEvent) {
  if (
    e.key === "ArrowUp" ||
    e.key === "ArrowDown" ||
    e.key === "ArrowLeft" ||
    e.key === "ArrowRight"
  ) {
    e.preventDefault();
    keys.add(e.key);
  }
}
```

```
function onKeyUp(e: KeyboardEvent) {
  if (
    e.key === "ArrowUp" ||
    e.key === "ArrowDown" ||
    e.key === "ArrowLeft" ||
    e.key === "ArrowRight"
  ) {
    e.preventDefault();
    keys.delete(e.key);
  }
}
```

```
let last = performance.now();
let frameId: number;
```

```
const loop = (now: number) => {
  const dt = now - last;
  last = now;
```

```
  const lobbyId = lobbyIdRef.current;
  if (lobbyId && keys.size > 0) {
    let vx = 0;
    let vy = 0;
```

```
    if (keys.has("ArrowUp")) vy -= 1;
```

```
if (keys.has("ArrowDown")) vy += 1;
if (keys.has("ArrowLeft")) vx -= 1;
if (keys.has("ArrowRight")) vx += 1;

const len = Math.hypot(vx, vy);
if (len > 0) {
  vx /= len;
  vy /= len;

  socket.emit("player:move", {
    lobbyId,
    dx: vx * MOVE_SPEED * dt,
    dy: vy * MOVE_SPEED * dt,
  });
}

frameId = requestAnimationFrame(loop);
};

frameId = requestAnimationFrame(loop);

window.addEventListener("keydown", onKeyDown);
window.addEventListener("keyup", onKeyUp);

return () => {
  window.removeEventListener("keydown", onKeyDown);
  window.removeEventListener("keyup", onKeyUp);
  cancelAnimationFrame(frameId);
  keys.clear();
};
}, [inMatch]);

function createLobby() {
  if (!nickname.trim()) {
    setError("Bitte Nickname eingeben");
    return;
  }
  socket.emit("lobby:create", { nickname: nickname.trim(), maxPlayers: 4 });
}

function joinLobby() {
  if (!nickname.trim() || !lobbyId.trim()) {
    setError("Lobby-ID und Nickname nötig");
    return;
  }
  socket.emit("lobby:join", {
    lobbyId: lobbyId.trim().toUpperCase(),
    nickname: nickname.trim(),
  });
}

function toggleReady() {
  if (!lobby) return;
  socket.emit("lobby:ready", {
```



```
    lobbyId: lobby.id,  
    ready: !(me?.ready ?? false),  
  });  
}  
  
function leaveLobby() {  
  if (!lobby) return;  
  socket.emit("lobby:leave", { lobbyId: lobby.id });  
  setLobby(null);  
  setMatchState(null);  
  setWinnerScreen(null);  
}  
  
function startMatch() {  
  if (!lobby) return;  
  socket.emit("match:start", { lobbyId: lobby.id });  
}  
  
function closeWinnerScreen() {  
  setWinnerScreen(null);  
}  
  
return (  
  <div className="screen">  
    { /* Error-Toast */ }  
    { error && (  
      <div className="toast toast-error" onClick={() => setError(null)}>  
        <span>⚠</span>  
        <p>{error}</p>  
      </div>  
    ) }  
  ) }  
  
  { /* Killfeed oben rechts */ }  
  { visibleKillFeed.length > 0 && (  
    <div className="killfeed">  
      { visibleKillFeed.map((item) => (  
        <div key={item.id} className="killfeed-item">  
          {item.text}  
        </div>  
      )) }  
    </div>  
  ) }  
  
  <div className="frame">  
    <header className="header">  
      <div className="logo">  
        <div className="logo-icon">  </div>  
        <div className="logo-text">  
          <h1>BOMB.IO</h1>  
          <p>Multiplayer Action Lobby</p>  
        </div>  
      </div>  
      <div className="header-status">  
        <span className="status-dot" />  
        <span>Online</span>  
      </div>  
    </header>  
  </div>  
)
```

```
</div>
</header>

{/* Winner-Overlay (Fullscreen) */}
{winnerScreen && (
  <WinnerOverlay
    state={winnerScreen}
    onClose={closeWinnerScreen}
    isOwner={!isOwner}
    onRestart={startMatch}
  />
)}

{!inMatch && (
  <main className="layout">
    {/* Linke Spalte: kurzer Hero */}
    <section className="hero">
      <h2>Schnelle Bomben-Matches mit deinen Freunden.</h2>
      <p>
        Erstelle eine Lobby, teile den Code und rennt, bevor die Bombe
        hochgeht.
      </p>
      <ul className="hero-list">
        <li>🏆 Live-Multiplayer im Browser</li>
        <li>🔥 1–4 Spieler, kurze Runden</li>
      </ul>
      <div className="hero-hint">
        Step&nbsp;1: Nickname eingeben · Step&nbsp;2: Lobby erstellen
        oder beitreten.
      </div>
    </section>

    {/* Rechte Spalte: Panels */}
    <section className="panels">
      {/* Start / Join Panel */}
      <div className="panel panel-grid">
        <div className="panel-column">
          <h3>Neues Spiel starten</h3>
          <label className="field">
            <span>Nickname</span>
            <input
              className="input"
              placeholder="Wie sollen dich die anderen sehen?"
              value={nickname}
              onChange={(e) => setNickname(e.target.value)}
            />
          </label>
          <button
            className="btn btn-primary btn-large"
            onClick={createLobby}
          >
            + Lobby erstellen
          </button>
        </div>
      </div>
    </section>
  </main>
)
```

```
<div className="panel-column">
  <h3>Einer Lobby beitreten</h3>
  <label className="field">
    <span>Lobby-ID</span>
    <input
      className="input"
      placeholder="z. B. QPC6GQ"
      value={lobbyId}
      onChange={(e) =>
        setLobbyId(e.target.value.toUpperCase())
      }
    />
  </label>
  <button className="btn btn-outline btn-large" onClick={joinLobby}>
    Beitreten
  </button>
  <p className="panel-hint">
    Den Code bekommst du von jemandem, der eine Lobby erstellt
    hat.
  </p>
</div>
</div>

{/* Aktive Lobby */}
{lobby && (
  <div className="panel panel-lobby">
    <div className="panel-lobby-header">
      <h3>Aktive Lobby</h3>
      <button
        className="pill copy-pill"
        onClick={() => navigator.clipboard.writeText(lobby.id)}
      >
        ID kopieren
      </button>
    </div>

    <div className="lobby-meta">
      <div>
        <span className="meta-label">Lobby-ID</span>
        <span className="code">{lobby.id}</span>
      </div>
      <div>
        <span className="meta-label">Spieler</span>
        <span>
          {lobby.players.length}/{lobby.maxPlayers}
        </span>
      </div>
      <div>
        <span className="meta-label">Runde</span>
        <span className="pill pill-timer">
          {lobby.round || 0}
        </span>
      </div>
    </div>

    <ul className="player-list">
```

```
{lobby.players.map((p) => (  
  <li key={p.id} className="player-item">  
    <div className="player-main">  
      <span className="avatar">  
        {p.nickname.substring(0, 1).toUpperCase()}  
      </span>  
      <span className="nickname">{p.nickname}</span>  
      {p.id === socket.id && (  
        <span className="tag tag-me">Du</span>  
      )}  
      {p.id === lobby.ownerId && (  
        <span className="tag tag-owner">Host</span>  
      )}  
      {!p.alive && (  
        <span className="tag tag-wait">Out</span>  
      )}  
    </div>  
    <span  
      className={  
        "tag " + (p.ready ? "tag-ready" : "tag-wait")  
      }  
    >  
      {p.ready ? "Bereit" : "Wartet"}  
    </span>  
  </li>  
))}  
</ul>  
  
<div className="lobby-actions">  
  <button  
    className="btn btn-primary"  
    onClick={toggleReady}  
  >  
    {me?.ready ? "Nicht bereit" : "Bereit"}  
  </button>  
  <button  
    className="btn btn-ghost"  
    onClick={leaveLobby}  
  >  
    Lobby verlassen  
  </button>  
  <button  
    className="btn btn-accent"  
    onClick={startMatch}  
    disabled={!isOwner}  
  >  
    Match starten{isOwner ? "" : " (Host)}  
  </button>  
</div>  
</div>  
))  
</section>  
</main>  
))  
  
{inMatch && matchState && (  

```

```
<main className="layout layout-match">
  <section className="panel panel-match-header">
    <div className="match-meta">
      <div>
        <span className="meta-label">Lobby</span>
        <span className="code">{matchState.lobbyId}</span>
      </div>
      <div>
        <span className="meta-label">Restzeit</span>
        <span className="pill pill-timer">
          {Math.ceil(matchState.remainingMs / 1000)}s
        </span>
      </div>
      <div>
        <span className="meta-label">Rolle</span>
        <span className="pill">
          {isSpectator ? "Spectator" : "Spieler"}
        </span>
      </div>
    </div>
  </section>

  <MatchView state={matchState} />
</main>
))

{inMatch && isSpectator && (
  <div className="spectator-banner">
    Du bist raus – du schaust jetzt als Spectator zu.
  </div>
)}
</div>
</div>
);
}

function MatchView({ state }: { state: MatchState }) {
  const width = 400;
  const height = 300;

  const bombSeconds =
    state.bomb != null
      ? Math.ceil(state.bomb.remainingMs / 1000)
      : null;

  return (
    <section className="panel panel-match">
      <div className="field-wrapper">
        <div className="field" style={{ width, height }}>
          {state.players.map((p) => {
            const isMe = p.id === socket.id;
            const isBombHolder = state.bomb?.holderId === p.id;
            const isCritical =
              isBombHolder && state.bomb && state.bomb.remainingMs <= 5000;
```

```
const classes = [
  "player-dot",
  isMe ? "player-dot-me" : "",
  isBombHolder ? "player-dot-bomb" : "",
  isCritical ? "pulse" : "",
]
.filter(Boolean)
.join(" ");

return (
  <div
    key={p.id}
    className={classes}
    style={{ left: p.x, top: p.y }}
  >
    {isBombHolder && bombSeconds !== null && (
      <span className="bomb-timer">{bombSeconds}</span>
    )}
  </div>
);
}}
</div>
</div>

<div className="field-footer">
  <span className="hint">Bewege dich mit den Pfeiltasten.</span>
  <span className="hint">Spieler: {state.players.length}</span>
</div>
</section>
);
}
```

```
function WinnerOverlay({
  state,
  onClose,
  isOwner,
  onRestart,
}: {
  state: WinnerScreenState;
  onClose: () => void;
  isOwner: boolean;
  onRestart: () => void;
}) {
  const meld = socket.id;

  return (
    <div className="winner-overlay">
      <div className="winner-card">
        <h2 className="winner-title">Runde beendet</h2>
        {state.winnerName ? (
          <p className="winner-main">
            Gewinner:{" "}
            <span className="winner-name">
              {state.winnerName}
            </span>
          </p>
        ) : null}
      </div>
    </div>
  );
}
```

```
        {state.winnerId === meld ? " (Du)" : ""}  
      </span>  
    </p>  
  ) : (  
    <p className="winner-main">Runde beendet</p>  
  )}  
  
  <h3>Platzierungen</h3>  
  <ol className="winner-list">  
    {state standings.map((s) => (  
      <li key={s.id}>  
        <span>  
          #{s.place} – {s.nickname}  
          {s.id === meld ? " (Du)" : ""}  
        </span>  
        <span className="winner-wins">Wins: {s.wins}</span>  
      </li>  
    ))}  
  </ol>  
  
  <div className="winner-actions">  
    <button className="btn btn-ghost" onClick={onClose}>  
      Schliessen  
    </button>  
    <button  
      className="btn btn-accent"  
      onClick={onRestart}  
      disabled={!isOwner}  
    >  
      Neue Runde starten{isOwner ? "" : " (Host)}  
    </button>  
  </div>  
</div>  
</div>  
);  
}
```

5.1.4 Socket.ts

```
import { io } from "socket.io-client";

const baseUrl = import.meta.env.DEV
  ? "http://localhost:3001"      // Dev → direkt Backend
  : "https://bombio.notascam.ch"; // Produktion → Domain, kein Port

export const socket = io(baseUrl, {
  transports: ["websocket"],
  path: "/socket.io/",
  withCredentials: false,
});
```


5.1.5 App.css

```
.screen {  
  width: 100%;  
  display: flex;  
  justify-content: center;  
  align-items: stretch;  
}
```

```
.frame {  
  width: 100%;  
  max-width: 1240px;  
  min-height: calc(100vh - 96px);  
  margin: auto;  
  padding: 26px 26px 24px;  
  border-radius: 28px;  
  background: linear-gradient(180deg, #050816 0%, #050714 60%, #040713 100%);  
  box-shadow:  
    0 0 0 2px rgba(0, 245, 255, 0.3),  
    0 26px 60px rgba(0, 0, 0, 0.9);  
  position: relative;  
  overflow: hidden;  
}
```

```
.frame::before {  
  content: "";  
  position: absolute;  
  inset: 8px;  
  border-radius: 22px;  
  box-shadow:  
    0 0 26px rgba(0, 245, 255, 0.7),  
    0 0 80px rgba(255, 138, 42, 0.3);  
  border: 2px solid rgba(0, 245, 255, 0.6);  
  opacity: 0.36;  
  pointer-events: none;
```

```
}
```

```
/* Header */
```

```
.header {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  background: radial-gradient(circle at top, #050814 0, #050713 70%);  
  padding: 14px 18px;  
  border-radius: 18px;  
  border: 2px solid rgba(0, 245, 255, 0.4);  
  box-shadow:  
    0 0 18px rgba(0, 245, 255, 0.5),  
    0 12px 26px rgba(0, 0, 0, 0.8);  
}
```

```
.logo {  
  display: flex;  
  gap: 12px;  
  align-items: center;  
}
```

```
.logo-icon {  
  width: 44px;  
  height: 44px;  
  background: radial-gradient(circle at 30% 20%, #fff7ed, #ff8a2a);  
  border-radius: 12px;  
  box-shadow:  
    0 0 20px rgba(255, 138, 42, 0.9),  
    0 0 2px #050714;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  animation: logoPulse 2.8s ease-in-out infinite;  
}
```

```
@keyframes logoPulse {
  50% {
    transform: translateY(-2px) scale(1.06);
  }
}

.logo-text h1 {
  margin: 0;
  font-family: "Orbitron", sans-serif;
  font-size: 1.6rem;
  letter-spacing: 0.18em;
  color: var(--neon-cyan-soft);
  text-shadow:
    0 0 16px rgba(0, 245, 255, 0.9),
    0 0 40px rgba(0, 245, 255, 0.8);
}

.logo-text p {
  margin: 2px 0 0;
  font-size: 0.78rem;
  letter-spacing: 0.16em;
  text-transform: uppercase;
  color: var(--neon-yellow);
  text-shadow: 0 0 10px rgba(255, 232, 59, 0.8);
}

.player-dot {
  position: absolute;
  width: 18px;
  height: 18px;
  margin-left: -9px;
  margin-top: -9px;
  border-radius: 50%;
  background: #f9fafb;
  display: flex;
```

```
align-items: center;
justify-content: center;
box-shadow:
  0 0 0 2px #050714,
  0 0 18px rgba(249, 250, 251, 0.7);
}

.player-dot-me {
background: var(--neon-magenta);
box-shadow:
  0 0 0 2px #050714,
  0 0 20px rgba(255, 60, 249, 0.95);
}

/* Bombe */
.player-dot-bomb {
background: var(--neon-yellow);
box-shadow:
  0 0 0 2px #050714,
  0 0 30px rgba(255, 232, 59, 1),
  0 0 40px rgba(255, 232, 59, 0.6);
}

.bomb-timer {
font-size: 0.65rem;
font-weight: 700;
color: #020617;
line-height: 1;
}

.player-hitbox {
position: absolute;
width: 40px;
height: 40px;
margin-left: -20px;
margin-top: -20px;
background: rgba(255,255,255,0.06);
```

```
border-radius: 50%;
pointer-events: none;
}
.player-dot-bomb.pulse {
  animation: bombPulse 0.4s infinite alternate;
}

@keyframes bombPulse {
  from {
    transform: scale(1);
    box-shadow: 0 0 30px rgba(255, 232, 59, 0.8);
  }
  to {
    transform: scale(1.25);
    box-shadow: 0 0 60px rgba(255, 60, 60, 1);
  }
}

.killfeed {
  position: fixed;
  top: 16px;
  right: 16px;
  width: 260px;
  display: flex;
  flex-direction: column;
  gap: 6px;
  z-index: 50;
}

.killfeed-item {
  padding: 6px 10px;
  border-radius: 10px;
  background: rgba(220, 38, 38, 0.16);
  border: 1px solid rgba(220, 38, 38, 0.6);
  color: #fecaca;
  font-size: 0.8rem;
  box-shadow: 0 0 12px rgba(220, 38, 38, 0.7);
}
```

```
animation: killfeedFade 6s forwards;  
}
```

```
@keyframes killfeedFade {  
  0% { opacity: 1; transform: translateX(0); }  
  90% { opacity: 1; }  
  100% { opacity: 0; transform: translateX(40px); }  
}
```

```
.winner-overlay {  
  position: fixed;  
  inset: 0;  
  background: rgba(0,0,0,0.82);  
  backdrop-filter: blur(6px);  
  z-index: 200;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

```
.winner-box {  
  padding: 40px 60px;  
  border-radius: 20px;  
  background: linear-gradient(180deg, #0a0c18, #050713);  
  border: 2px solid var(--neon-cyan);  
  box-shadow:  
    0 0 30px rgba(0, 245, 255, 0.8),  
    0 0 60px rgba(255, 60, 249, 0.4);  
  text-align: center;  
}
```

```
.winner-title {  
  font-family: "Orbitron";  
  font-size: 2rem;  
  letter-spacing: 0.15em;  
  color: var(--neon-cyan);  
  text-shadow:
```

```
    0 0 20px rgba(0, 245, 255, 1),  
    0 0 40px rgba(0, 245, 255, 0.8);  
margin-bottom: 14px;  
}
```

```
.winner-name {  
  font-size: 1.4rem;  
  color: var(--neon-magenta);  
  margin-bottom: 20px;  
}
```

```
.winner-actions button {  
  margin: 10px 12px;  
  padding: 10px 22px;  
  border-radius: 12px;  
  cursor: pointer;  
  font-size: 0.9rem;  
  text-transform: uppercase;  
  border: 2px solid var(--neon-cyan);  
  background: #020617;  
  box-shadow: 0 0 14px rgba(0, 245, 255, 0.4);  
}
```

```
.spectator-banner {  
  position: fixed;  
  bottom: 20px;  
  left: 50%;  
  transform: translateX(-50%);  
  padding: 6px 14px;  
  background: rgba(148, 163, 184, 0.18);  
  border: 1px solid rgba(148, 163, 184, 0.5);  
  border-radius: 12px;  
  color: #e5e7eb;  
  font-size: 0.9rem;  
  text-shadow: 0 0 10px rgba(148, 163, 184, 0.8);  
  z-index: 80;  
}
```


5.1.6 Index.css

```
:root {
  --bg-main: #040713;
  --bg-panel: #070b1c;
  --bg-panel-soft: #0b1024;

  --neon-cyan: #00f5ff;
  --neon-cyan-soft: #4bfbff;
  --neon-magenta: #ff3cf9;
  --neon-orange: #ff8a2a;
  --neon-yellow: #ffe83b;
  --neon-border: #00f5ff;

  --text-main: #f9fafb;
  --text-muted: #9ca3af;

  font-family: system-ui, -apple-system, BlinkMacSystemFont, "Inter", sans-serif;
  line-height: 1.4;
  font-weight: 400;
  color: var(--text-main);
  background-color: #000;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
}

*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  display: flex;
```

```
justify-content: center;
align-items: stretch;
background:
  radial-gradient(circle at top, #101b3c 0, #050814 40%, #000 90%),
  radial-gradient(circle at 10% 80%, rgba(0, 245, 255, 0.15) 0, transparent 55%),
  radial-gradient(circle at 90% 20%, rgba(255, 138, 42, 0.18) 0, transparent 55%);
color: var(--text-main);
overflow-y: auto;
}
```

```
#root {
  flex: 1;
  display: flex;
  justify-content: center;
  align-items: stretch;
  padding: 32px 16px;
}
```

```
@media (min-width: 960px) {
  #root {
    padding: 40px 32px;
  }
}
```

```
a {
  color: inherit;
  text-decoration: none;
}
a:hover {
  text-decoration: underline;
}
```

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
```

}

}

5.1.7 Index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>client</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Anhang B: Testcode Unit-Tests

Im Rahmen dieses Projekts wurden keine automatisierten Unit-Tests implementiert. Die Anwendung wurde stattdessen mithilfe der definierten Testfälle aus dem Testkonzept (Funktions-, Integrations- und Szenariotests) manuell überprüft.